Resolución del Problema de los Filósofos Comensales usando Semáforos

Curso: Sistemas Operativos

Docente: Fabio Andres Guzman Figueroa

Trabajo 1: Sincronización con Semáforos

Estudiante: Santiago Hernández Morantes

Fecha: 25 de febrero de 2026

1. Introducción

En sistemas concurrentes, múltiples procesos o hilos pueden competir por recursos compartidos.

Cuando no existe un mecanismo de sincronización adecuado, pueden presentarse problemas como:

- Condiciones de carrera
- Interbloqueo (deadlock)
- Inanición (starvation)

Los semáforos, propuestos por Edsger Dijkstra, son una herramienta fundamental para controlar el acceso a recursos compartidos y evitar estos problemas.

En este trabajo se implementa el clásico problema de los Filósofos Comensales, utilizando semáforos POSIX en C.

2. Descripción del Problema

El problema plantea:

- N filósofos sentados alrededor de una mesa circular.
- Entre cada par de filósofos hay un cubierto.
- Para comer, un filósofo necesita dos cubiertos: El de su izquierda y el de su derecha.

Cada filósofo alterna entre:

- Pensar
- Tener hambre
- Comer

El reto es garantizar que:

- No haya interbloqueo (deadlock).
- No haya uso simultáneo de un mismo cubierto.
- El sistema sea concurrente (no estrictamente secuencial).

3. Condiciones del Deadlock

Para que ocurra un deadlock deben cumplirse cuatro condiciones:

1. **Exclusión mutua:** Los cubiertos no pueden compartirse.
2. **Hold and wait:** Un filósofo puede tener un cubierto y esperar otro.
3. **No apropiación:** No se puede forzar a soltar un cubierto.
4. **Espera circular:** Cada filósofo espera el recurso del siguiente.

Nuestra solución rompe la condición de espera circular mediante una verificación controlada del estado de los vecinos.

4. Estrategia de Solución

Se implementó la solución clásica de Dijkstra basada en:

- Un semáforo global `mutex` para proteger la sección crítica.
- Un semáforo privado `s[i]` por cada filósofo.
- Un arreglo `state[i]` que puede ser: `THINKING`, `HUNGRY` o `EATING`.

Idea principal:

Un filósofo solo puede comer si:

- Está hambriento.
- Su vecino izquierdo no está comiendo.
- Su vecino derecho no está comiendo.

Si no cumple estas condiciones, queda bloqueado en su semáforo privado hasta que pueda continuar. Esto permite comer en paralelo si no son vecinos, evitar interbloqueo y maximizar concurrencia.

5. Esquema del Algoritmo

Estados:

THINKING → HUNGRY → EATING → THINKING

Funciones clave:

- `take_forks(i)`

- put_forks(i)
- test(i)

6. Pseudocódigo Explicado

```
función test(i):

si estado[i] == HAMBRIENTO

y estado[izquierdo(i)] != COMIENDO

y estado[derecho(i)] != COMIENDO entonces

estado[i] = COMIENDO

signal(s[i])



función tomar_cubiertos(i):

wait(mutex)

estado[i] = HAMBRIENTO

test(i)

signal(mutex)

wait(s[i])



función dejar_cubiertos(i):

wait(mutex)

estado[i] = PENSANDO
```

```
test(izquierdo(i))

test(derecho(i))

signal(mutex)
```

7. Justificación de Corrección

No hay uso simultáneo del mismo cubierto: Un filósofo solo entra en estado EATING si ambos vecinos NO están comiendo.

No hay deadlock: Nunca se permite que todos los filósofos queden esperando circularmente, ya que el acceso está controlado por la verificación del estado.

Hay concurrencia real: Filósofos no vecinos pueden comer simultáneamente. Esto se evidencia en la salida del programa donde múltiples filósofos comen en bloques temporales similares.

8. Implementación en C

📂 trabajo1.c

```c
#define _DEFAULT_SOURCE // expone usleep/useconds_t con -std=c11 -pedantic

#include  <pthread.h>

#include  <semaphore.h>

#include  <stdio.h>

#include  <stdlib.h>

#include  <unistd.h>

#include  <time.h>
```

```
#ifndef N_PHILOS
#define N_PHILOS 5
#endif

#ifndef ROUNDS
#define ROUNDS 7
#endif

// Tiempos fijos (microsegundos) para TODOS los hilos
#ifndef THINK_US
#define THINK_US 700000
#endif

#ifndef EAT_US
#define EAT_US 700000
#endif

typedef  enum { THINKING, HUNGRY, EATING } state_t;

static state_t state[N_PHILOS];
```

```c
static sem_t mutex; // protege state[]
static sem_t s[N_PHILOS]; // semáforo privado por filósofo


static pthread_t th[N_PHILOS];
static  int ids[N_PHILOS];


// Para medir tiempo relativo (ms desde inicio)
static  struct timespec t0;


static  inline  int LEFT(int i) { return (i + N_PHILOS - 1) % N_PHILOS; }
static  inline  int RIGHT(int i) { return (i + 1) % N_PHILOS; }


static  long now_ms_since_start(void) {
struct timespec t;
clock_gettime(CLOCK_MONOTONIC, &t);
long sec = (long)(t.tv_sec - t0.tv_sec);
long nsec = (long)(t.tv_nsec - t0.tv_nsec);
return sec * 1000L + nsec / 1000000L;
}


// Imprime en formato uniforme para que se vea "pro"
static  void log_event(int i, int round, const  char *msg) {
```

```
// Filósofo se imprime 1..N
printf("[%6ld ms] F[%d] R[%d/%d] %s\n",
now_ms_since_start(), i + 1, round, ROUNDS, msg);
fflush(stdout);
}


// Si i está hambriento y sus vecinos no están comiendo, habilítalo a comer
static  void test(int i) {
if (state[i] == HUNGRY &&
state[LEFT(i)] != EATING &&
state[RIGHT(i)] != EATING) {


state[i] = EATING;
// Despierta SOLO a este filósofo
sem_post(&s[i]);
}
}


static  void take_forks(int i, int round) {
sem_wait(&mutex);
```

```
state[i] = HUNGRY;
log_event(i, round, "hambriento...");


test(i);


sem_post(&mutex);


// Si no quedó habilitado, se bloquea hasta que alguien lo despierte
sem_wait(&s[i]);


// Ya quedó en EATING
char buf[128];
snprintf(buf, sizeof(buf), "usa cubiertos (%d, %d) y come...",
i + 1, RIGHT(i) + 1);
log_event(i, round, buf);
}


static  void put_forks(int i, int round) {
sem_wait(&mutex);


state[i] = THINKING;
```

```
char buf[128];
snprintf(buf, sizeof(buf), "regresa los cubiertos (%d, %d)",
i + 1, RIGHT(i) + 1);
log_event(i, round, buf);


// Al soltar, intenta habilitar a los vecinos
test(LEFT(i));
test(RIGHT(i));


sem_post(&mutex);
}


static  void *philosopher(void *arg) {
int i = *(int *)arg;


for (int r = 1; r <= ROUNDS; r++) {
log_event(i, r, "piensa...");
usleep((useconds_t)THINK_US);


take_forks(i, r);
usleep((useconds_t)EAT_US);
```

```
put_forks(i, r);
}

// Mensaje final por hilo
printf("[%6ld ms] F[%d] FIN: completo %d rondas.\n",
now_ms_since_start(), i + 1, ROUNDS);
fflush(stdout);

return NULL;
}

int main(void) {
// Inicio del reloj
clock_gettime(CLOCK_MONOTONIC, &t0);

// Mensaje inicial bonito
printf("=== Filosofos comensales | N=%d | RONDAS=%d | THINK_US=%d | EAT_US=%d ===\r
N_PHILOS, ROUNDS, THINK_US, EAT_US);
fflush(stdout);

// Inicializa mutex y semáforos privados
if (sem_init(&mutex, 0, 1) != 0) { perror("sem_init(mutex)"); return  1; }
```

```
for (int i = 0; i < N_PHILOS; i++) {
state[i] = THINKING;
if (sem_init(&s[i], 0, 0) != 0) { perror("sem_init(s[i])"); return  1; }
}

// Crea hilos con IDs estables
for (int i = 0; i < N_PHILOS; i++) {
ids[i] = i;
if (pthread_create(&th[i], NULL, philosopher, &ids[i]) != 0) {
perror("pthread_create");
return  1;
}
}

for (int i = 0; i < N_PHILOS; i++) pthread_join(th[i], NULL);

for (int i = 0; i < N_PHILOS; i++) sem_destroy(&s[i]);
sem_destroy(&mutex);

printf("\n=== Listo: todos los filosofos completaron %d rondas. ===\n", ROUNDS);
```

```
return 0;

}
```

9. Evidencia de Ejecución

Salida ajustada a solo una ronda para que se visualice bien y quepa en la terminal

```
~/Doc/College/2026/SistemasOperativos/T/trabajo1  main ?1  ./trabajo1
=== Filosofos comensales | N=5 | RONDAS=1 | THINK_US=700000 | EAT_US=700000 ===

[     0 ms] F[1] R[1/1] piensa...
[     0 ms] F[2] R[1/1] piensa...
[     0 ms] F[3] R[1/1] piensa...
[     0 ms] F[4] R[1/1] piensa...
[     0 ms] F[5] R[1/1] piensa...
[   701 ms] F[1] R[1/1] hambriento...
[   701 ms] F[1] R[1/1] usa cubiertos (1, 2) y come...
[   701 ms] F[2] R[1/1] hambriento...
[   701 ms] F[3] R[1/1] hambriento...
[   701 ms] F[3] R[1/1] usa cubiertos (3, 4) y come...
[   701 ms] F[4] R[1/1] hambriento...
[   701 ms] F[5] R[1/1] hambriento...
[  1401 ms] F[1] R[1/1] regresa los cubiertos (1, 2)
[  1401 ms] F[1] FIN: completo 1 rondas.
[  1401 ms] F[3] R[1/1] regresa los cubiertos (3, 4)
[  1401 ms] F[3] FIN: completo 1 rondas.
[  1401 ms] F[5] R[1/1] usa cubiertos (5, 1) y come...
[  1401 ms] F[2] R[1/1] usa cubiertos (2, 3) y come...
[  2102 ms] F[5] R[1/1] regresa los cubiertos (5, 1)
[  2102 ms] F[5] FIN: completo 1 rondas.
[  2102 ms] F[2] R[1/1] regresa los cubiertos (2, 3)
[  2102 ms] F[2] FIN: completo 1 rondas.
[  2102 ms] F[4] R[1/1] usa cubiertos (4, 5) y come...
[  2802 ms] F[4] R[1/1] regresa los cubiertos (4, 5)
[  2802 ms] F[4] FIN: completo 1 rondas.

=== Listo: todos los filosofos completaron 1 rondas. ===
~/Doc/College/2026/SistemasOperativos/T/trabajo1  main ?1
```

10. Análisis de Resultados

Se observa que:

- Todos los filósofos ejecutan 7 rondas correctamente.
- No se presentan bloqueos.
- La ejecución se da en bloques de tiempo similares debido a tiempos uniformes.
- Se evidencia concurrencia real mediante timestamps.

11. Conclusión

Se logró implementar correctamente el problema de los Filósofos Comensales utilizando semáforos POSIX.

La solución evita interbloqueo, permite ejecución concurrente y sincroniza correctamente los accesos a recursos. Es configurable mediante macros para cambiar número de filósofos y rondas. Este ejercicio demuestra el uso práctico de semáforos para el control de sincronización en sistemas concurrentes.